

INFORME TECNICO DEFINITIVO DEL PROYECTO OSPEDALE

Universidad del Norte

Departamento de Ingenieria de Sistemas y Computacion

Programacion Orientada a Objetos

Parcial 3

1. INTRODUCCION Y ARQUITECTURA GENERAL DEL PROYECTO

1.1 Proposito del Proyecto

Ospedale es un sistema academico de gestion hospitalaria desarrollado como ejercicio integrador de Programacion Orientada a Objetos. El dominio funcional del sistema gira alrededor de tres perfiles de usuario: administrador, paciente y doctor. A partir de estos tres actores, el sistema debe soportar operaciones de autentificacion, administracion de usuarios, solicitud y gestion de citas, solicitud y aprobacion de hospitalizaciones, prescripcion de medicamentos y visualizacion ordenada de la informacion clinica u operativa en componentes graficos.

Desde una perspectiva academica, el objetivo del parcial no era solamente "hacer que la interfaz funcione", sino someter un proyecto existente a una refactorizacion controlada hacia una arquitectura MVC apoyada en principios SOLID. En otras palabras, el trabajo exigia convertir una base de codigo con acoplamientos fuertes, logica mezclada y comportamiento repartido de manera informal, en un sistema con separacion clara entre vista, controlador, modelo, reglas de negocio, persistencia simulada y mecanismos de comunicacion entre capas.

La migracion a una arquitectura mas limpia tiene un impacto tecnico y pedagogico muy importante:

- Reduce el acoplamiento entre la interfaz grafica y la logica del negocio.
- Hace visible el flujo real de datos entre actores, vistas, controladores y modelos.
- Permite validar requisitos del parcial de forma trazable y demostrable.
- Facilita la sustentacion, porque cada decision de diseno puede justificarse por responsabilidad, cohesion, extensibilidad y mantenibilidad.
- Sienta bases para patrones de diseno adicionales, como Repository, Factory, Singleton y Observer.

En el estado refactorizado del proyecto, el sistema ya no se entiende como "una aplicacion Swing con codigo incrustado", sino como un conjunto de capas con contratos mas claros:

- La vista captura eventos y renderiza informacion.
- El controlador traduce la intencion de la vista a un caso de uso.
- El servicio aplica las reglas del negocio.
- El repositorio abstrae el acceso al almacenamiento simulado.
- El almacenamiento centraliza carga, serializacion, ordenamiento, ids y observadores.

Esta organizacion no solamente ayuda a cumplir la rubrica, sino que transforma el proyecto en un ejemplo mucho mas cercano a una aplicacion mantenible de la vida real.

1.2 Arquitectura General Aplicada

La solucion final se organizo en torno a una arquitectura MVC ampliada con capas auxiliares:

- view: interfaz Swing y coordinacion de eventos de usuario.
- controller: puerta de entrada para los casos de uso que la vista invoca.
- service: capa donde viven las reglas funcionales del hospital.

- model: entidades del dominio.
- repository: interfaces e implementaciones para acceso a datos.
- validator: reglas de validacion de entrada y consistencia.
- factory: construccion centralizada de modelos cuando aporta claridad.
- observer: desacoplamiento reactivo entre el almacenamiento y las vistas.
- model/storage: almacenamiento simulado, carga JSON, serializacion y ordenamientos.

El resultado es una arquitectura donde la vista no valida formatos complejos, el controlador no contiene reglas pesadas, el modelo no conoce detalles de interfaz y la persistencia simulada se encapsula en un componente dedicado.

1.3 Estructura de Carpetas del Proyecto Refactorizado

El siguiente arbol resume la estructura relevante del proyecto refactorizado. Se omiten los artefactos compilados build/classes por ser productos generados automaticamente y no fuentes authored del equipo.

```
Ospedale/
  ■■■ README.md
  ■■■ build.xml
  ■■■ manifest.mf
  ■■■ .gitignore
  ■■■ docs/
  ■ ■■■ Informe_Rubrica_Parcial3_Ospedale.md
  ■ ■■■ Informe_Rubrica_Parcial3_Ospedale.pdf
  ■ ■■■ Informe_Tecnico_Definitivo_Ospedale.md
  ■■■ json/
  ■ ■■■ users.json
  ■■■ lib/
  ■ ■■■ flatlaf-demo-3.6.jar
  ■ ■■■ json-20250107.jar
  ■■■ nbproject/
  ■ ■■■ build-impl.xml
  ■ ■■■ genfiles.properties
  ■ ■■■ project.properties
  ■ ■■■ project.xml
  ■ ■■■ private/
  ■ ■■■ private.properties
  ■ ■■■ private.xml
  ■■■ src/
  ■■■ main/
  ■ ■■■ Main.java
  ■■■ package/ospedale/
  ■■■ controller/
  ■ ■■■ AppointmentController.java
  ■ ■■■ AuthController.java
  ■ ■■■ DoctorController.java
  ■ ■■■ MedicalServiceController.java
  ■ ■■■ PatientController.java
  ■ ■■■ utils/
  ■ ■■■ Response.java
  ■ ■■■ Status.java
  ■■■ factory/
  ■ ■■■ PatientFactory.java
  ■■■ model/
  ■ ■■■ Administrator.java
  ■ ■■■ Appointment.java
  ■ ■■■ AppointmentStatus.java
  ■ ■■■ Doctor.java
  ■ ■■■ Hospitalization.java
  ■ ■■■ HospitalizationStatus.java
  ■ ■■■ Patient.java
  ■ ■■■ Prescription.java
  ■ ■■■ RoomType.java
  ■ ■■■ Specialty.java
```

```

■ ■■■ User.java
■ ■■■ storage/
■ ■■■ Storage.java
■■■ observer/
■ ■■■ StorageEventType.java
■ ■■■ StorageObserver.java
■■■ repository/
■ ■■■ AppointmentRepository.java
■ ■■■ AppointmentRepositoryImpl.java
■ ■■■ PatientRepository.java
■ ■■■ PatientRepositoryImpl.java
■■■ service/
■ ■■■ AppointmentService.java
■ ■■■ AuthService.java
■ ■■■ DoctorService.java
■ ■■■ HospitalizationService.java
■ ■■■ PatientService.java
■■■ validator/
■ ■■■ AppointmentValidator.java
■ ■■■ DoctorValidator.java
■ ■■■ PatientValidator.java
■ ■■■ UserValidator.java
■■■ view/
■■■ Admin_View.form
■■■ Admin_View.java
■■■ Doctor_View.form
■■■ Doctor_View.java
■■■ Login.form
■■■ Login.java
■■■ PanelRound.java
■■■ Patient_View.form
■■■ Patient_View.java

```

1.4 Diccionario y Razon de Ser de Cada Archivo

En esta seccion se documentan todos los archivos authored y relevantes para el proyecto. No se incluyen uno por uno los .class de build/classes, porque son artefactos generados por compilacion y no forman parte del diseno fuente mantenible del sistema.

1.4.1 Archivos Raiz

****README.md****

Es el punto minimo de documentacion del repositorio. Su papel no es operativo dentro del runtime del sistema, pero si es indispensable para la entrega academica, porque ahi deben quedar consignados los integrantes y el contexto general del proyecto. Sin este archivo, el repositorio queda huérfano de contexto institucional y se incumple una exigencia expresa de entrega.

****build.xml****

Es el descriptor principal de Ant para compilar, ejecutar y empaquetar el proyecto NetBeans. Aunque no contiene reglas de negocio, es indispensable porque define el mecanismo reproducible de construccion del sistema. En un entorno academico, este archivo garantiza que el proyecto pueda compilarse desde la estructura esperada por el IDE o por Ant.

****manifest.mf****

Es el archivo de manifiesto asociado al empaquetado Java. Su existencia es importante para la construccion del artefacto ejecutable y la definicion de metadatos del proyecto. Aunque es pequeno, actua como parte del puente entre codigo fuente y artefacto final.

****gitignore****

Controla que archivos temporales o generados no se conviertan en ruido de versionamiento. Es importante para mantener el repositorio limpio, evitar commits accidentales de artefactos irrelevantes y estabilizar el trabajo colaborativo.

1.4.2 Carpeta docs

****docs/Informe_Rubrica_Parcial3_Ospedale.md****

Fue el primer informe tecnico de soporte generado durante el proceso de trabajo. Sirve como antecedente documental y como base de trazabilidad sobre la evolucion del analisis del sistema frente a la rubrica.

****docs/Informe_Rubrica_Parcial3_Ospedale.pdf****

Es la exportacion del informe preliminar en formato portable. No interviene en la ejecucion del sistema, pero si tiene valor academico y de sustentacion, porque encapsula un snapshot de documentacion ya listo para compartir.

****docs/Informe_Tecnico_Definitivo_Ospedale.md****

Es el presente documento tecnico definitivo. Se convierte en la pieza central de sustentacion arquitectonica, funcional y academica del proyecto. Su razon de ser es demostrar, con trazabilidad exhaustiva, como se satisfacen los requisitos del parcial y como se justifica el diseno.

1.4.3 Carpeta json

****json/users.json****

Es la fuente inicial de datos del sistema. Su importancia es fundamental, porque evita hardcodear usuarios dentro del codigo y permite simular persistencia realista en memoria. Desde este archivo se crean administradores, pacientes y doctores al iniciar el sistema.

1.4.4 Carpeta lib

****lib/flatlaf-demo-3.6.jar****

Contiene la libreria de look and feel utilizada para estilizar la interfaz Swing. Es importante porque desacopla la apariencia visual del sistema del estilo Swing por defecto y permite que el proyecto arranque con una apariencia consistente.

****lib/json-20250107.jar****

Contiene la libreria org.json, utilizada para parsear el archivo users.json. Es indispensable porque la rubrica exige cargar informacion desde JSON y esta libreria es precisamente el medio tecnico para hacerlo sin construir un parser manual.

1.4.5 Carpeta nbproject

****nbproject/build-impl.xml****

Es un archivo generado por NetBeans con reglas de build internas. No se suele editar manualmente, pero es necesario para que el proyecto se integre correctamente con el ecosistema del IDE.

****nbproject/genfiles.properties****

Registra metadatos de archivos generados dentro de la configuracion de NetBeans. Su importancia radica en que ayuda a que el IDE sincronice su estado interno con la estructura del proyecto.

****nbproject/project.properties****

Contiene propiedades del proyecto como rutas de compilacion, clase principal, encoding y bibliotecas. Es parte del contrato de configuracion del build y, por tanto, esencial para reproducibilidad.

****nbproject/project.xml****

Declara la identidad estructural del proyecto dentro de NetBeans. Aunque no tiene logica de negocio, es parte del esqueleto de configuracion sin el cual el proyecto no se abre correctamente en el entorno esperado.

****nbproject/private/private.properties****

Almacena preferencias privadas del entorno local. No es funcional para el negocio, pero si forma parte de la experiencia de compilacion y apertura del proyecto en el equipo del desarrollador.

****nbproject/private/private.xml****

Complementa la configuracion privada del IDE. Su razon de ser es netamente de entorno, no de dominio, pero sigue siendo un archivo del proyecto en terminos de estructura operativa.

1.4.6 Arranque del Sistema

****src/main/Main.java****

Es el punto de entrada formal del sistema. Su papel es desacoplar la ejecucion de la aplicacion respecto de las vistas, inicializar el look and feel y lanzar la ventana de login. Es indispensable para cumplir el requisito de que las vistas no se autoejecuten.

1.4.7 Capa controller

****src/packagee/ospedale/controller/AppointmentController.java****

Es un controlador legado orientado a operaciones directas de cita. En la arquitectura final, parte de su funcionalidad fue absorbida conceptualmente por MedicalServiceController, pero se conserva como fachada puntual para operaciones basicas de citas. Su razon de ser actual es mantener compatibilidad y separacion semantica de ciertos accesos.

****src/packagee/ospedale/controller/AuthController.java****

Es la fachada de autenticacion que la vista de login invoca. No contiene la logica de negocio del login; su responsabilidad es exponer un punto de entrada limpio para la vista y delegar a AuthService.

****src/packagee/ospedale/controller/DoctorController.java****

Expone a la vista los casos de uso relacionados con doctores: registrar, actualizar, consultar y listar. Su importancia radica en servir como frontera MVC entre la interfaz grafica y los servicios de dominio del doctor.

****src/packagee/ospedale/controller/MedicalServiceController.java****

Es el controlador funcional mas importante del proyecto. Actua como puerta de entrada para los casos de uso de citas y hospitalizaciones, pero en la version refactorizada ya no concentra toda la logica sino que delega a AppointmentService y HospitalizationService.

****src/packagee/ospedale/controller/PatientController.java****

Es la fachada para registrar, actualizar y consultar pacientes. Su importancia esta en encapsular el acceso desde la vista a PatientService, evitando que la UI conozca detalles del almacenamiento o la validacion.

1.4.8 controller/utls

****src/packagee/ospedale/controller/utls/Response.java****

Define la estructura uniforme de respuesta que los controladores devuelven a la vista. Es indispensable porque materializa uno de los requisitos mas importantes del parcial: toda operacion debe responder con mensaje, estado y, cuando aplique, datos serializados.

****src/packagee/ospedale/controller/utls/Status.java****

Centraliza codigos de estado utilizados en las respuestas. Su existencia evita numeros magicos repartidos por el sistema y fortalece la legibilidad de los flujos de exito, error de validacion, no encontrado o error interno.

1.4.9 factory

****src/packagee/ospedale/factory/PatientFactory.java****

Centraliza la construccion de instancias Patient a partir de datos crudos provenientes de la interfaz. Es importante porque encapsula detalles de parseo y conversion, reduciendo la responsabilidad del servicio.

1.4.10 model

****src/packagee/ospedale/model/User.java****

Es la abstraccion base de todos los usuarios del sistema. Su razon de ser es definir atributos y comportamiento comun como id, username, nombres y password, habilitando reutilizacion y jerarquia.

****src/packagee/ospedale/model/Administrator.java****

Representa al administrador del sistema. Es una especializacion de User cuya importancia radica en formalizar el rol con el que se administran doctores y se navega hacia vistas de terceros.

****src/packagee/ospedale/model/Patient.java****

Modela al paciente y almacena sus datos personales y relaciones con citas y hospitalizaciones. Es indispensable porque concentra el estado de un actor principal del dominio.

****src/packagee/ospedale/model/Doctor.java****

Modela al doctor, incluyendo especialidad, licencia, oficina y relaciones con citas y hospitalizaciones. Es clave para sostener reglas como disponibilidad, aprobacion y prescripcion.

****src/packagee/ospedale/model/Appointment.java****

Representa la entidad de cita medica. Su importancia es central porque encapsula el paciente, doctor, fecha y hora, motivo, tipo, estado y resultados clinicos de la atencion.

****src/packagee/ospedale/model/AppointmentStatus.java****

Define los posibles estados de una cita. Es indispensable para expresar de forma segura el ciclo de vida REQUESTED, PENDING, COMPLETED y CANCELED.

****src/packagee/ospedale/model/Hospitalization.java****

Representa una hospitalizacion y vincula paciente, doctor, fecha, razon, tipo de habitacion, observaciones y estado. Es la entidad que soporta el segundo gran flujo del sistema ademas de las citas.

****src/packagee/ospedale/model/HospitalizationStatus.java****

Define los estados de hospitalizacion. Su razon de ser es evitar cadenas literales y modelar formalmente el ciclo REQUESTED, ONGOING y CANCELED.

****src/packagee/ospedale/model/Prescription.java****

Modela una prescripcion asociada a una cita. Su importancia es permitir que el doctor agregue uno o varios medicamentos a una consulta en curso.

****src/packagee/ospedale/model/RoomType.java****

Enumera tipos de habitacion hospitalaria. Es clave para normalizar la seleccion de habitaciones y evitar inconsistencias textuales.

****src/packagee/ospedale/model/Specialty.java****

Enumera las especialidades medicas del sistema. Su importancia es enorme porque soporta asignacion por especialidad, visualizacion y validaciones semanticas.

1.4.11 model/storage

****src/packagee/ospedale/model/storage/Storage.java****

Es el nucleo de persistencia simulada del proyecto. Carga usuarios desde JSON, mantiene las colecciones vivas en memoria, genera ids, serializa modelos hacia la vista, ordena resultados, verifica disponibilidad y publica eventos del observador. Es probablemente la clase de infraestructura mas importante del sistema.

1.4.12 observer

****src/packagee/ospedale/observer/StorageObserver.java****

Define la abstraccion de observador para reaccionar a cambios en el almacenamiento. Es indispensable para el bono de actualizacion automatica de tablas.

****src/packagee/ospedale/observer/StorageEventType.java****

Define el tipo de evento publicado por Storage. Su importancia radica en distinguir si cambio la coleccion de usuarios, la de citas o la de hospitalizaciones.

1.4.13 repository

****src/packagee/ospedale/repository/PatientRepository.java****

Es la interfaz que expresa las operaciones necesarias para trabajar con pacientes desde el servicio. Su razon de ser es reducir acoplamiento entre la logica del negocio y la implementacion del almacenamiento.

****src/packagee/ospedale/repository/PatientRepositoryImpl.java****

Implementa PatientRepository usando Storage. Es indispensable porque traduce la abstraccion del repositorio a las operaciones concretas del almacenamiento simulado.

****src/packagee/ospedale/repository/AppointmentRepository.java****

Es la interfaz que define el contrato de acceso para citas. Su utilidad es sostener el desacoplamiento entre servicio de citas y mecanismo de almacenamiento.

****src/packagee/ospedale/repository/AppointmentRepositoryImpl.java****

Implementa AppointmentRepository delegando en Storage. Es importante porque permite al servicio de citas trabajar con una interfaz especializada y no con una clase monolitica.

1.4.14 service

****src/packagee/ospedale/service/AuthService.java****

Contiene la logica de autenticacion del sistema. Es indispensable porque encapsula las reglas de login, validacion minima de credenciales y serializacion del usuario autenticado.

****src/packagee/ospedale/service/PatientService.java****

Contiene reglas de registro, actualizacion y consulta de pacientes. Su importancia esta en centralizar validaciones, unicidad y publicacion de eventos de cambio.

****src/packagee/ospedale/service/DoctorService.java****

Concentra las reglas para registrar, actualizar y consultar doctores. Es el lugar donde se validan id, username, licencia, oficina y especialidad.

****src/packagee/ospedale/service/AppointmentService.java****

Es la clase mas rica en logica funcional. Contiene la creacion de citas, su aceptacion, completion, cancelacion, reagendamiento, prescripcion, consultas ordenadas y hospitalizacion desde cita.

****src/packagee/ospedale/service/HospitalizationService.java****

Agrupar reglas de solicitud, aprobacion, cancelacion y consulta de hospitalizaciones. Su separacion respecto de AppointmentService permite mejorar cohesion semantica del dominio.

1.4.15 validator

****src/packagee/ospedale/validator/UserValidator.java****

Centraliza las reglas comunes de usuario: validacion de id, username, password y nombres. Es una pieza muy importante para evitar validaciones duplicadas entre pacientes y doctores.

****src/packagee/ospedale/validator/PatientValidator.java****

Contiene reglas particulares del paciente: email, telefono y fecha de nacimiento. Es indispensable para aislar detalles de formato fuera de la vista y del servicio principal.

****src/packagee/ospedale/validator/DoctorValidator.java****

Contiene reglas especificas del doctor: formato de licencia y de oficina. Su razon de ser es expresar el contrato formal exigido por la rubrica.

****src/packagee/ospedale/validator/AppointmentValidator.java****

Modela ciertas validaciones puntuales de cita, como formato de fecha o razon. Aunque parte de la logica de tiempo fue centralizada despues en AppointmentService, sigue siendo un apoyo de validacion especializada.

1.4.16 view

****src/packagee/ospedale/view/Login.java****

Es la vista de autenticacion y registro inicial de pacientes. Su importancia radica en ser la puerta de entrada del sistema y el primer punto donde se aplica el patron MVC: captura datos, invoca controladores y reacciona a respuestas.

****src/packagee/ospedale/view/Login.form****

Es la definicion visual del formulario Swing de login/registro. Es indispensable para preservar el layout exigido por el parcial sin reescribir la interfaz desde cero.

****src/packagee/ospedale/view/Admin_View.java****

Es la vista del administrador. Permite registrar doctores y navegar hacia vistas de pacientes o doctores como usuario supervisor. Su importancia funcional es enorme porque materializa el rol mas privilegiado del sistema.

****src/packagee/ospedale/view/Admin_View.form****

Preserva el layout de la vista de administrador. Es importante para cumplir la restriccion de no alterar visualmente la interfaz base.

****src/packagee/ospedale/view/Patient_View.java****

Es la vista del paciente, donde se visualizan datos, historial de citas, solicitud de citas, solicitud de hospitalizacion y cancelacion de citas. Su razon de ser es materializar el flujo de paciente dentro del patron MVC.

****src/packagee/ospedale/view/Patient_View.form****

Es el descriptor visual Swing de la vista de paciente. Conserva el esqueleto visual y permite mantener el compromiso de no mover componentes del layout.

****src/packagee/ospedale/view/Doctor_View.java****

Es la vista del doctor. Desde aqui se aceptan citas, se completan, se reagendan, se prescriben medicamentos, se consultan citas de pacientes y se gestionan hospitalizaciones. Es la vista mas compleja por densidad funcional.

****src/packagee/ospedale/view/Doctor_View.form****

Es la contraparte visual generada de la vista del doctor. Su razon de ser es mantener el layout exigido mientras el codigo Java se reorienta a una mejor separacion de responsabilidades.

****src/packagee/ospedale/view/PanelRound.java****

Es un componente visual reutilizable para la apariencia de paneles redondeados. No contiene logica de negocio, pero es importante para la consistencia de la interfaz.

1.5 Resumen Arquitectonico del Estado Final

Tras la refactorizacion, Ospedale no es simplemente un conjunto de ventanas con botones. Es un sistema con:

- arranque desacoplado (Main)
- fachada de controladores
- logica de negocio segmentada por servicio
- validacion especializada
- acceso a datos abstraído
- almacenamiento centralizado y observable
- vistas reactivas que operan sobre datos serializados

Esta base es suficientemente robusta para sostener la sustentacion del parcial en terminos de diseño, trazabilidad y cumplimiento funcional.

2. REFACTORIZACION MVC Y CUMPLIMIENTO DE REQUISITOS (MAPEO 1 A 1)

A. Capa de Vistas (View)

A.i Requisito: Aspecto visual intacto

****Sentido del requisito.**** El parcial prohíbe agregar nuevos componentes o reorganizar arbitrariamente la interfaz. La refactorización debía concentrarse en arquitectura y comportamiento, no en rediseño visual.

****Solución aplicada.**** El proyecto mantuvo los formularios .form generados por NetBeans como base del layout. Las clases Java asociadas (Login.java, Admin_View.java, Patient_View.java, Doctor_View.java) fueron ajustadas en comportamiento y conexión con controladores, pero no se replanteó el diseño gráfico desde cero.

****Referencias de implementación.****

- src/packagee/ospedale/view/Login.form
- src/packagee/ospedale/view/Admin_View.form
- src/packagee/ospedale/view/Patient_View.form
- src/packagee/ospedale/view/Doctor_View.form

El bloque initComponents() de cada vista conserva la estructura generada del formulario, lo cual garantiza que la refactorización recaerá en lógica y no en layout.

A.ii Requisito: Renombramiento de componentes

****Sentido del requisito.**** La rubrica exige nombres de componentes más claros para que el formulario y el código sean legibles.

****Estado en el proyecto.**** Este punto se cumple de forma fuerte en varios componentes clave (username_login_input, password_login_input, select_attending_doctor, select_patient, select_appointment, back_button, logout_button, save_button, etc.), pero la aplicación conserva algunos identificadores heredados del constructor de formularios de NetBeans en controles secundarios y contenedores internos, especialmente en Patient_View y partes de Doctor_View.

****Justificación técnica.**** Durante la refactorización se priorizó el renombramiento de componentes interactivos cuya lectura afecta la mantenibilidad del flujo funcional. Los contenedores de infraestructura y algunas etiquetas generadas (jLabel..., panel..., jTextArea...) siguen siendo deuda técnica de naming, pero no afectan el comportamiento ni la separación de responsabilidades.

****Referencias de implementación.****

- src/packagee/ospedale/view/Login.java - campos de autenticación y registro con nombres semánticos.
- src/packagee/ospedale/view/Admin_View.java - componentes como select_doctor, select_patient, doctor_view_button.
- src/packagee/ospedale/view/Doctor_View.java - componentes clave como select_appointment, select_patient, select_requests, medication_name_input.

- `src/packagee/ospedale/view/Patient_View.java` - componentes semanticos como `select_attending_doctor`, `select_desired_room_type`, `select_id_appointment`.

A.iii Requisito: Sin verificaciones de datos en la vista

****Sentido del requisito.**** La vista no debe decidir reglas de negocio ni formatos canonicos. Debe capturar, delegar y renderizar respuestas.

****Solucion aplicada.**** Las vistas coleccionan cadenas y selecciones de usuario y las envian a controladores, los cuales delegan a servicios y validadores. La verificacion de ids, username, password, email, phone, licencia, oficina, fecha, hora y reglas de negocio reside fuera de la UI.

****Referencias principales.****

- `src/packagee/ospedale/view/Login.java:463 y :503`
- `src/packagee/ospedale/view/Patient_View.java:904, :963, :994`
- `src/packagee/ospedale/view/Doctor_View.java:1263, :1341, :1374, :1409, :1451`

****Capa que absorbe la validacion.****

- `src/packagee/ospedale/validator/UserValidator.java`
- `src/packagee/ospedale/validator/PatientValidator.java`
- `src/packagee/ospedale/validator/DoctorValidator.java`
- `src/packagee/ospedale/service/AppointmentService.java:392 y :401`

****Observacion de rigor.**** Existen verificaciones ligeras de seleccion nula en algunas vistas de administrador para evitar errores de navegacion (Please select a doctor, Please select a patient). Estas no constituyen validaciones del dominio clinico ni del formato de entrada, sino guardas de interaccion del formulario.

A.iv Requisito: Flujo y enrutamiento de usuarios

A.iv.1 Administrador -> Vista Administrador

Se cumple en `src/packagee/ospedale/view/Login.java:503-537`, metodo `enter_login1ActionPerformed`. Cuando `AuthController.login` retorna el rol `admin`, la vista crea una instancia de `Admin_View` y navega hacia ella.

A.iv.2 Paciente -> Vista Paciente

Tambien se cumple en `Login.enter_login1ActionPerformed`. Si el rol retornado es `patient`, se construye `Patient_View(id, false)`.

A.iv.3 Doctor -> Vista Doctor

Se cumple en el mismo metodo. Si el rol es `doctor`, se crea `Doctor_View(id, false)`.

A.iv.4 Logout -> Vista Login

Se cumple en:

- src/packagee/ospedale/view/Admin_View.java - logout_buttonActionPerformed
- src/packagee/ospedale/view/Patient_View.java - logout_buttonActionPerformed
- src/packagee/ospedale/view/Doctor_View.java - logout_buttonActionPerformed

Todas estas operaciones abren new Login() y cierran la vista actual con dispose().

A.iv.5 El administrador accede a las vistas de paciente y doctor

Se cumple en:

- src/packagee/ospedale/view/Admin_View.java:495-505 - doctor_view_buttonActionPerformed
- src/packagee/ospedale/view/Admin_View.java:513-523 - patient_view_buttonActionPerformed

En ambos casos se pasa el id del usuario seleccionado y el flag true para indicar que se accedio desde el administrador.

A.iv.6 El boton Back solo para administrador

Se materializa en los constructores:

- src/packagee/ospedale/view/Patient_View.java:39-48
- src/packagee/ospedale/view/Doctor_View.java:38-47

En ambos casos la visibilidad se controla mediante fromAdmin.

A.v Requisito: Carga automatica de datos del usuario autenticado

****Paciente.**** src/packagee/ospedale/view/Patient_View.java:73-89, metodo loadPatientInfo, consulta PatientController.getPatientInfo y carga firstname, lastname, birthdate, email, phone, address, username, password y genero.

****Doctor.**** src/packagee/ospedale/view/Doctor_View.java:76-87, metodo loadDoctorInfo, consulta DoctorController.getDoctorInfo y carga firstname, lastname, licence, office, specialty, username y password.

Esto demuestra que la vista no adivina el estado del usuario: lo consulta formalmente a traves de la capa controladora.

A.vi Requisito: Carga automatica y dependiente de ComboBoxes

Se cumple mediante metodos dedicados que consultan datos serializados y repueblan combos:

- Admin_View.loadDoctorsComboBox - src/packagee/ospedale/view/Admin_View.java:59
- Admin_View.loadPatientsComboBox - src/packagee/ospedale/view/Admin_View.java:71
- Patient_View.loadDoctorsComboBox - src/packagee/ospedale/view/Patient_View.java:111

- Patient_View.loadAppointmentsComboBox - src/packagee/ospedale/view/Patient_View.java:125
- Patient_View.loadRoomTypesComboBox - src/packagee/ospedale/view/Patient_View.java
- Doctor_View.loadAppointmentsComboBoxes - src/packagee/ospedale/view/Doctor_View.java:101
- Doctor_View.loadPatientsComboBox - src/packagee/ospedale/view/Doctor_View.java:120
- Doctor_View.loadHospitalizationsComboBox - src/packagee/ospedale/view/Doctor_View.java:134

Adicionalmente, existe carga dependiente de seleccion en:

- Patient_View.selectSpecialtyActionPerformed
- Patient_View.selectDoctorActionPerformed

Cuando se elige especialidad, el combo se llena con especialidades. Cuando se elige doctor, se recarga con ids de doctores.

A.vii Requisito: Invocacion de controladores

La vista interactua siempre contra la capa controladora, no contra Storage ni contra los modelos. Ejemplos:

- Registro de paciente: Login.java:463 -> PatientController.registerPatient
- Login: Login.java:503 -> AuthController.login
- Registro de doctor: Admin_View.java:474 -> DoctorController.registerDoctor
- Actualizacion de paciente: Patient_View.java:904 -> PatientController.updatePatient
- Solicitud de cita: Patient_View.java:963 -> MedicalServiceController.requestAppointment
- Solicitud de hospitalizacion: Patient_View.java:994 -> MedicalServiceController.requestHospitalization
- Aceptar cita: Doctor_View.java:1341 -> MedicalServiceController.acceptAppointment
- Completar cita: Doctor_View.java:1374 -> MedicalServiceController.completeAppointment
- Prescribir: Doctor_View.java:1409 -> MedicalServiceController.prescribeMedication

A.viii Requisito: Notificar al usuario el resultado

Esto se cumple mediante JOptionPane y, en el caso de Login, con el helper showResponse:

- src/packagee/ospedale/view/Login.java:30
- src/packagee/ospedale/view/Admin_View.java
- src/packagee/ospedale/view/Patient_View.java
- src/packagee/ospedale/view/Doctor_View.java

La UI distingue errores de servidor (≥ 500), errores de cliente/validacion (≥ 400) y respuestas exitosas.

A.ix Requisito: Limpiar campos cuando la operacion es exitosa

Se cumple mediante rutinas de limpieza y reinicializacion de componentes:

- Login.clearPatientRegisterFields
- Login.clearLoginFields
- limpieza tras crear cita en Patient_View.java:971-977
- limpieza tras solicitar hospitalizacion en Patient_View.java:1001-1007
- limpieza tras reagendar, aceptar o completar en Doctor_View.java:1348-1350, :1390-1393, :1458-1462

A.x Requisito: Archivo Main independiente

Se cumple en src/main/Main.java:12-19. La aplicacion arranca desde main, se configura FlatDarkLaf y luego se ejecuta new Login().setVisible(true). Ninguna vista declara un main propio como punto oficial de arranque del sistema.

B. Capa de Controladores y Casos de Uso del Parcial

En esta seccion se hace el mapeo uno a uno de los requisitos funcionales expresados por el enunciado. En casi todos los casos, el controlador es una fachada delgada y la logica real vive en servicios o validadores; por ello se referencia tanto el controlador como el archivo de negocio correspondiente.

B.1 Los doctores son registrados por el usuario administrador

****Solucion.**** El registro de doctor se expone en Admin_View, no en la vista de login ni en la del paciente. El flujo termina en DoctorController.registerDoctor y luego en DoctorService.registerDoctor.

****Referencias.****

- src/packagee/ospedale/view/Admin_View.java:474
- src/packagee/ospedale/controller/DoctorController.java:8
- src/packagee/ospedale/service/DoctorService.java:21

B.2 El administrador puede realizar operaciones como paciente y/o doctor

****Solucion.**** El administrador navega hacia Patient_View y Doctor_View con el id seleccionado y un flag fromAdmin=true. Esto le permite operar en dichas vistas con los mismos casos de uso disponibles para el rol funcional.

****Referencias.****

- src/packagee/ospedale/view/Admin_View.java:495 y :513

B.3 IDs unicos, mayores que 0, de 12 digitos y no modificables

****Solucion.**** El formato se valida en UserValidator.validateUserId. La unicidad se verifica en PatientService.registerPatient y DoctorService.registerDoctor. La no modificacion se garantiza porque las operaciones de update reciben el id existente como referencia y no ofrecen cambio del atributo id del modelo.

****Referencias.****

- src/packagee/ospedale/validator/UserValidator.java
- src/packagee/ospedale/service/PatientService.java:78
- src/packagee/ospedale/service/DoctorService.java:25
- src/packagee/ospedale/model/User.java

B.4 Username unico y modificable

****Solucion.**** La unicidad se valida consultando repository.getUserByUsername o storage.getUserByUsername antes de crear o actualizar. El username es mutable a traves de setUsername.

****Referencias.****

- PatientService.registerPatient y updatePatient
- DoctorService.registerDoctor y updateDoctor
- src/packagee/ospedale/model/User.java

B.5 Password y confirmacion deben coincidir

****Solucion.**** Se centraliza en UserValidator.validatePassword, reutilizado por pacientes y doctores.

****Referencias.****

- src/packagee/ospedale/validator/UserValidator.java
- src/packagee/ospedale/service/PatientService.java:43
- src/packagee/ospedale/service/DoctorService.java:140

B.6 Telefono del paciente de exactamente 10 digitos

****Solucion.**** PatientValidator.validatePhone.

****Referencia.****

- src/packagee/ospedale/validator/PatientValidator.java

B.7 Email del paciente valido

****Solucion.**** PatientValidator.validateEmail.

****Referencia.****

- src/packagee/ospedale/validator/PatientValidator.java

B.8 Fecha de nacimiento valida y formato AAAA-MM-DD

****Solucion.**** PatientValidator.validateBirthdate.

****Referencia.****

- src/packagee/ospedale/validator/PatientValidator.java

B.9 Licencia del doctor en formato L-XXXXXXXXXX MTL

****Solucion.**** DoctorValidator.validateLicence.

****Referencia.****

- src/packagee/ospedale/validator/DoctorValidator.java

B.10 Oficina del doctor en formato O-XXX

****Solucion.**** DoctorValidator.validateOffice.

****Referencia.****

- src/packagee/ospedale/validator/DoctorValidator.java

B.11 Actualizar paciente con los mismos requerimientos de creacion

****Solucion.**** PatientService.updatePatient reutiliza validatePatientData, el mismo validador compuesto usado en registro.

****Referencia.****

- src/packagee/ospedale/service/PatientService.java:26
- src/packagee/ospedale/service/PatientService.java:115

B.12 Actualizar doctor con los mismos requerimientos de creacion

****Solucion.**** DoctorService.updateDoctor reutiliza validateDoctorData, el mismo bloque usado en alta.

****Referencia.****

- src/packagee/ospedale/service/DoctorService.java:54
- src/packagee/ospedale/service/DoctorService.java:127

B.13 Las citas duran 15 minutos

****Solucion de dominio.**** El proyecto asume que la unidad minima de agenda es un bloque de 15 minutos. Esta regla se refleja indirectamente en la validacion de hora y en la politica de disponibilidad, donde solo se admiten minutos 00, 15, 30 o 45.

****Referencia.****

- src/packagee/ospedale/service/AppointmentService.java:401

B.14 ID automatico de cita A-{id_paciente}-NNNN

****Solucion.**** Storage.generateAppointmentId.

****Referencia.****

- src/packagee/ospedale/model/storage/Storage.java

El contador es por paciente y se incrementa progresivamente.

B.15 Fecha de cita valida

****Solucion.**** AppointmentService.validateDate.

****Referencia.****

- src/packagee/ospedale/service/AppointmentService.java:392

B.16 Hora de cita valida y minutos en cuartos de hora

****Solucion.**** AppointmentService.validateTime.

****Referencia.****

- src/packagee/ospedale/service/AppointmentService.java:401

B.17 Toda operacion que requiera doctor garantiza doctor valido

****Solucion.**** Antes de crear citas u hospitalizaciones, o al operar sobre ellas, el servicio consulta Storage.getDoctorById y valida nulidad.

****Referencias.****

- AppointmentService.requestAppointment
- HospitalizationService.requestHospitalization
- Storage.getDoctorById

B.18 Solicitud de cita con doctor especifico y validacion de disponibilidad

****Solucion.**** La rama byDoctor de AppointmentService.requestAppointment verifica que el doctor exista, que tenga disponibilidad y que la especialidad sea consistente cuando aplique.

****Referencia.****

- `src/packagee/ospedale/service/AppointmentService.java:88`

B.19 Solicitud de cita por especialidad con asignacion automatica

****Solucion.**** Si no se envia doctor, el servicio busca un doctor disponible por especialidad mediante `Storage.findAvailableDoctor`.

****Referencias.****

- `AppointmentService.requestAppointment`
- `Storage.findAvailableDoctor`

B.20 Todas las citas empiezan en REQUESTED

****Solucion.**** El constructor de `Appointment` inicializa `status = AppointmentStatus.REQUESTED`.

****Referencia.****

- `src/packagee/ospedale/model/Appointment.java`

B.21 Toda accion sobre cita garantiza cita valida

****Solucion.**** Los servicios de aceptar, completar, cancelar, reagendar y prescribir consultan primero `Storage.getAppointmentById`.

****Referencias.****

- `AppointmentService.acceptAppointment`
- `AppointmentService.completeAppointment`
- `AppointmentService.cancelAppointment`
- `AppointmentService.rescheduleAppointment`
- `AppointmentService.prescribeMedication`

B.22 Un doctor acepta una cita y pasa a PENDING

****Solucion.**** `AppointmentService.acceptAppointment`.

****Referencia.****

- `src/packagee/ospedale/service/AppointmentService.java:162`

B.23 Un doctor completa una cita y pasa a COMPLETED

****Solucion.**** AppointmentService.completeAppointment.

****Referencia.****

- src/packagee/ospedale/service/AppointmentService.java:183

B.24 Un paciente cancela cita no COMPLETED y pasa a CANCELED

****Solucion.**** AppointmentService.cancelAppointment bloquea cancelacion de COMPLETED y cambia a CANCELED en los demas casos.

****Referencia.****

- src/packagee/ospedale/service/AppointmentService.java:209

B.25 Reagendar cita sin cambiar el dia, con razon agregada

****Solucion.**** AppointmentService.rescheduleAppointment conserva la fecha original, reemplaza solo la hora y concatena la razon nueva a la anterior.

****Referencia.****

- src/packagee/ospedale/service/AppointmentService.java:230

B.26 Prescribir una o varias medicaciones solo si la cita esta PENDING

****Solucion.**** AppointmentService.prescribeMedication verifica estado PENDING antes de crear Prescription.

****Referencia.****

- src/packagee/ospedale/service/AppointmentService.java:264

B.27 ID automatico de hospitalizacion H-{id_paciente}-NNNN

****Solucion.**** Storage.generateHospitalizationId.

****Referencia.****

- src/packagee/ospedale/model/storage/Storage.java

B.28 Fecha de hospitalizacion valida

****Solucion.**** HospitalizationService.requestHospitalization reutiliza AppointmentService.validateDate.

****Referencia.****

- `src/packagee/ospedale/service/HospitalizationService.java:24`

B.29 Hospitalizaciones del paciente empiezan en REQUESTED

****Solucion.**** El constructor base de Hospitalization asigna HospitalizationStatus.REQUESTED.

****Referencia.****

- `src/packagee/ospedale/model/Hospitalization.java`

B.30 Toda accion sobre hospitalizacion garantiza hospitalizacion valida

****Solucion.**** `acceptHospitalization` y `cancelHospitalization` consultan primero `Storage.getHospitalizationById`.

****Referencia.****

- `src/packagee/ospedale/service/HospitalizationService.java:60`
- `src/packagee/ospedale/service/HospitalizationService.java:81`

B.31 Un doctor aprueba hospitalizacion y pasa a ONGOING

****Solucion.**** `HospitalizationService.acceptHospitalization`.

B.32 Hospitalizacion directa desde cita

****Solucion.**** `AppointmentService.hospitalizeFromAppointment` completa la cita y crea una hospitalizacion con estado ONGOING.

****Referencia.****

- `src/packagee/ospedale/service/AppointmentService.java:288`

B.33 Un doctor deniega hospitalizacion y pasa a CANCELED

****Solucion.**** `HospitalizationService.cancelHospitalization`.

B.34 Citas del paciente ordenadas descendentemente

****Solucion.**** `Storage.getAppointmentsByPatientSorted`.

****Referencia.****

- `src/packagee/ospedale/model/storage/Storage.java:276`

B.35 Citas del doctor ordenadas descendentemente y con filtro total/pendientes

****Solucion.**** Storage.getAppointmentsByDoctorSorted.

****Referencia.****

- src/packagee/ospedale/model/storage/Storage.java:284

B.36 No enviar objetos del modelo a la vista; enviar serializacion

****Solucion.**** Storage.serializeUser, serializePatient, serializeDoctor, serializeAppointment y serializeHospitalization convierten los modelos a HashMap<String,Object>.

****Referencias.****

- src/packagee/ospedale/model/storage/Storage.java:327
- :345
- :356
- :365
- :380

C. Capa de Modelos y Persistencia

C.1 Diseñar modelos necesarios siguiendo SOLID

El proyecto define modelos especializados (User, Patient, Doctor, Appointment, Hospitalization, Prescription) y enums del dominio. Esta separacion impide un "modelo universal" caotico y favorece cohesiones claras por entidad.

C.2 Simulacion de almacenamiento

Se cumple mediante Storage, singleton en memoria que mantiene listas vivas de usuarios, citas y hospitalizaciones.

C.3 Carga de JSON y relaciones

Storage.loadUsersFromJson utiliza org.json para leer users.json y construir instancias concretas segun el campo type. Las relaciones posteriores entre pacientes, doctores, citas y hospitalizaciones se realizan al crear entidades en tiempo de ejecucion (addAppointment, constructores de Hospitalization, etc.).

3. APLICACION INTEGRAL DE LOS PRINCIPIOS SOLID

3.1 Single Responsibility Principle (SRP)

El sistema cumple SRP porque cada clase dominante tiene una unica razon principal de cambio:

- las vistas cambian si cambia la interaccion o la presentacion
- los controladores cambian si cambia la orquestacion del caso de uso
- los servicios cambian si cambian las reglas del dominio
- los validadores cambian si cambian formatos o restricciones
- los repositorios cambian si cambia el acceso a datos
- Storage cambia si cambia la estrategia de persistencia simulada o serializacion

Ejemplos didacticos:

- Login.java no decide si un username es unico; solo captura datos y muestra mensajes.
- PatientService.java no dibuja tablas ni abre ventanas; solo ejecuta casos de uso de paciente.
- DoctorValidator.java no sabe nada de Swing ni de serializacion; solo valida licencia y oficina.
- AppointmentRepositoryImpl.java no conoce reglas de disponibilidad; solo delega acceso a Storage.

3.2 Open/Closed Principle (OCP)

El sistema avanza hacia OCP porque los cambios funcionales se canalizan agregando o extendiendo servicios, validadores y enums sin obligar a reescribir la capa completa.

Ejemplos:

- la logica de hospitalizacion se encapsulo en HospitalizationService en lugar de seguir inflando un unico controlador
- el observador se agrego como mecanismo nuevo a traves de StorageObserver y StorageEventType sin reescribir las vistas desde cero
- las especialidades medicas viven en un enum ampliable (Specialty)

La idea clave es que el sistema no depende de ifs desperdigados por toda la interfaz, sino de puntos localizados de extension.

3.3 Liskov Substitution Principle (LSP)

LSP se evidencia en la jerarquia User -> Administrator, Patient, Doctor. En los lugares donde el sistema necesita trabajar con usuarios en general, cualquiera de las subclases puede reemplazar al tipo base sin romper invariantes fundamentales.

Ejemplos:

- Storage almacena ArrayList<User>

- `serializeUser` opera sobre `User` y luego especializa por subtipo
- la autenticacion devuelve un `User` desde `getUserByUsername`

Esto demuestra que el sistema reconoce un contrato comun de usuario y especializa solo donde el dominio realmente lo exige.

3.4 Interface Segregation Principle (ISP)

ISP se aplica especialmente en el nivel de repositorios:

- `PatientRepository` ofrece solo operaciones necesarias para pacientes
- `AppointmentRepository` ofrece solo operaciones necesarias para citas

El servicio de pacientes no depende de una interfaz gigante con operaciones de hospitalizaciones o prescripciones que no necesita. De igual manera, el servicio de citas usa su propio contrato especializado.

3.5 Dependency Inversion Principle (DIP)

La aplicacion de DIP es mas visible en la frontera servicio-repositorio que en la frontera vista-controlador. Los servicios de paciente y cita trabajan contra interfaces (`PatientRepository`, `AppointmentRepository`) y no dependen directamente de detalles internos del almacenamiento concreto.

Ejemplos:

- `PatientService` depende de `PatientRepository`
- `AppointmentService` depende de `AppointmentRepository`

En el nivel superior, los controladores aun delegan a servicios concretos mediante metodos estaticos, por lo que DIP no esta llevado a una inversion total por inyeccion de dependencias en toda la aplicacion. Sin embargo, el proyecto si muestra una aplicacion clara del principio en las capas donde mas impacto tiene para mantenibilidad academica.

4. DESARROLLO DE BONIFICACIONES (IMPLEMENTACION COMPLETA)

4.1 Bono 1: SOLID en los Controladores

La bonificacion sobre SOLID en controladores se cumple principalmente por desacoplamiento de responsabilidades. En la version original, gran parte de la logica de negocio estaba mezclada cerca de la UI o del controlador. En la version refactorizada:

- AuthController solo delega a AuthService
- PatientController solo delega a PatientService
- DoctorController solo delega a DoctorService
- MedicalServiceController solo delega a AppointmentService y HospitalizationService

Esto evita controladores gigantescos y permite que cada flujo de negocio tenga un hogar mas apropiado. El controlador deja de ser una clase procedural desbordada y pasa a ser una fachada de coordinacion.

Patrones y tecnicas usados para esto:

- ****Facade ligera en controladores**** para exponer casos de uso a la vista.
- ****Service Layer**** para concentrar reglas funcionales.
- ****Repository**** para aislar almacenamiento.
- ****Factory**** para centralizar construccion de Patient.

Aunque no se introdujo un arbol formal de Strategy o Command por cada accion, la separacion alcanzada si materializa el objetivo del bono: controladores delgados, mas cohesionados y alineados con SOLID.

4.2 Bono 2: Patron Observer para actualizacion automatica de tablas

Esta bonificacion si quedo implementada de forma explicita.

Subject

El Subject central es Storage:

- src/packagee/ospedale/model/storage/Storage.java

Este componente conoce la lista de observadores, permite suscripcion y desuscripcion, y emite eventos mediante publishEvent.

Metodos relevantes:

- addObserver - Storage.java:57
- removeObserver - Storage.java:63
- publishEvent - Storage.java:67

Observer

La interfaz observadora es:

- `src/packagee/ospedale/observer/StorageObserver.java`

Los tipos de evento son:

- `src/packagee/ospedale/observer/StorageEventType.java`

Observers concretos

Las vistas se suscriben al almacenamiento:

- `Admin_View.registerObserver - Admin_View.java:40`
- `Patient_View.registerObserver - Patient_View.java:51`
- `Doctor_View.registerObserver - Doctor_View.java:50`

Cada vista reacciona a eventos distintos:

- `USERS_CHANGED` -> combos y datos del usuario
- `APPOINTMENTS_CHANGED` -> tablas y combos de citas
- `HOSPITALIZATIONS_CHANGED` -> listas/combos de hospitalizaciones

Emision de eventos desde la capa de negocio

Los servicios publican cambios:

- `PatientService` publica `USERS_CHANGED`
- `DoctorService` publica `USERS_CHANGED`
- `AppointmentService` publica `APPOINTMENTS_CHANGED`
- `HospitalizationService` publica `HOSPITALIZATIONS_CHANGED`

Esto significa que, cuando se crea o modifica una cita, ya no hace falta depender unicamente de un boton "Refresh". El sistema tiene un canal reactivo formal entre el modelo persistente y la interfaz.

Valor arquitectonico del bono

Este patron no solo suma puntos por automatismo visual. Tambien demuestra:

- bajo acoplamiento entre almacenamiento y UI
- separacion clara entre cambio de estado y renderizado
- extensibilidad futura para mas vistas observadoras

5. CONCLUSION GENERAL

El proyecto Ospedale, en su estado refactorizado, muestra una transformacion real hacia una arquitectura mas disciplinada. El sistema cumple la mayor parte sustantiva de la rubrica a traves de:

- arranque desacoplado por Main
- vistas que delegan a controladores
- controladores delgados
- servicios con reglas de negocio
- validadores especializados
- repositorios por dominio
- almacenamiento simulado centralizado
- serializacion obligatoria de respuestas
- actualizacion automatica por observador

Desde una mirada de Arquitectura de Software y POO, el mayor valor del trabajo no esta solo en que "funcione", sino en que ahora es explicable, justificable y sustentable. Cada responsabilidad esta mucho mas localizada que antes, la lectura del proyecto es mas clara y el flujo entre capas es defendible frente a la rubrica del parcial.

Si este documento se usa como base de sustentacion, los archivos que mas conviene dominar son:

- Main.java
- AuthController.java y AuthService.java
- PatientController.java y PatientService.java
- DoctorController.java y DoctorService.java
- MedicalServiceController.java
- AppointmentService.java
- HospitalizationService.java
- Storage.java
- Login.java
- Admin_View.java
- Patient_View.java
- Doctor_View.java

En ellos se concentran casi todos los argumentos fuertes de arquitectura, cumplimiento funcional y justificacion de diseno.